

Splight: A Lightweight Block Cipher for Resource-Constrained Embedded Devices

Qian Zhou, Qing Ye, Chengzhe Lai, *Member, IEEE*, Yong Yu, *Fellow, IEEE*, Min Zhang, Xiaobing Liu

Abstract—The rapid expansion of Internet of Things (IoT) deployments has accelerated sensor infrastructure development. However, resource-constrained embedded devices require cryptographic algorithms that achieve an optimal balance between low power and high security. Given that the traditional Feistel structure suffers from diffusion inefficiency due to half-state updating. Furthermore, the SP structure has the problem of security defects due to the limited number of active S-boxes. In this paper, we propose an optimized logic combination method combining Feistel structure and S-P-S operation, and design a lightweight block cipher algorithm, called Splight. Splight overcomes the efficiency bottleneck of Feistel structure through full-state iteration and enhances active S-box numbers with the help of S-P-S substitution layer to strengthen security. Then, We construct a half-byte automated analysis model based on MILP and employ the Gurobi solver to achieve verifiable security evaluation. Meanwhile, we perform a security analysis on Splight to illustrate that the algorithm has high security performance. Finally, a round-based hardware architecture description and ASIC implementation of Splight are established. Comparison of experimental results shows that the encryption and decryption rate is 224.16Mbps and 386.57Mbps in software performance with high runtime efficiency. The hardware performance has low power and high throughput with 3.44mW and 1.14Gbps throughput in SMIC180nm.

Index Terms—Feistel structure, SP structure, MILP, Internet of Things, Lightweight block cipher, Hardware implementation cost.

I. INTRODUCTION

IN recent years, with the rapid advancement and wide application of Internet of Things (IoT) technology, there has been an explosive growth in the number of IoT devices [1]. Based on identification technology, pervasive computing and edge computing, information interaction constitutes the core of IoT. As the major carrier of information collection, IoT nodes contain data that often involves user privacy or sensitive information [2]. With the increase in the number of IoT devices,

their node privacy and security issues have been highlighted. For example, wireless networks have vulnerabilities that lead to data leakage or tampering [3]. The leakage of sensitive data, including personal identity details and location information, could lead to a significant erosion of public trust and create obstacles for IoT development. Consequently, integrating enhanced security measures into both hardware components and software systems has become imperative to ensure robust safeguards for transmitting, processing, and storing private data [4]. Scholarly efforts have increasingly focused on developing methodologies to secure sensitive information acquired by IoT nodes. In an attempt to address this challenge, Chai et al. [5] proposed a multi-key distribution scheme based on ECDH and public key authentication. This scheme generated a secure data block for each participant that contains their facial code and the public key from the owner to ensure key transfer security. Hu et al. [6] proposed a k-means clustering scheme for multiple IoT devices. The scheme reduces system overhead by optimizing the key management mechanism, ensures privacy and security for both participants and intermediate variables, while maintains high precision clustering performance. Hu et al. [7] proposed a data aggregation and trust management scheme based on smart contracts for IoT smart grids. This scheme strengthens the ability to resist differential attacks and safeguards against privacy leakage in the aggregation process.

Generally, data collected by IoT devices requires to be encrypted and backed up to ensure confidentiality and integrity. However, a large number of resource-constrained devices exist in the physical layer of IoT. They have weak computational power, small available storage space, and are mostly battery-powered with strict cost and energy requirements. Dalmaso et al. [9] point out that lightweight encryption techniques are crucial for securing constrained devices. Asymmetric cryptography is harder to lightweight due to different keys for encryption and decryption. Thus, most lightweight cryptography algorithms are symmetric. Advanced Encryption Standard (AES) [10] is the most used and researched block cipher algorithm. Its 128-bit version is suitable for lightweight devices. Over the decades, many scholars have designed block ciphers with superior performance, such as PRESENT [11], LBLOCK [12], SIMON, SPECK [13], PRINCE [14], RECTANGLE [15], and SKINNY [16]. Bogdanov et al. [11] designed the SPN-structured PRESENT algorithm. The encryption process employs 16 S-boxes in parallel to reduce hardware overhead. To further reduce hardware area, Wu et al. [12] designed the LBLOCK algorithm with Feistel structure, which has a key scheduling algorithm modeled from PRESENT with NLFSSR structure, and utilizes S-box transformation and cyclic shifting

This work was supported in part by the Defense Foundation Strengthening Technology Fund under Grant 2021-JCJQ-JJ-1267, in part by the National Natural Science Foundation of China (6227072896, U24B20149, 62272385, U23A20302, 62311540156), in part by the National Cryptologic Science Fund of China (2025NCSF02025), in part by the Key Research and Development Program of Shaanxi (2024GX-ZDCYL-01-09), and in part by the Mid-Aged and Young Science and Technology Innovation Leading Talent Program of Xi'an (25ZQRC00017). (Corresponding author: Chengzhe Lai)

Qian Zhou, Qing Ye, Min Zhang, Xiaobing Liu are with the Department of Information Security, Naval University of Engineering, Wuhan 430030, China (e-mail: D24381005@nue.edu.cn; dis_yeqing@sohu.com; zmm Patton@163.com; lxbdu@163.com).

Chengzhe Lai is with the college of cyberspace security, Xi'an university of Posts and Telecommunications, Xi'an 710121, China (e-mail: lcz_xupt@163.com).

Yong Yu is with the School of Computer Science, Shaanxi Normal University, Xi'an, 710119, China (e-mail: yuyong@snnu.edu.cn).

to generate the round key. Beaulieu et al. [13] constructed the SIMON algorithm with Feistel structure using a combination of SP structure and logic instructions. It has a simple circuit implementation and is efficient in both software and hardware environments.

The design of any cryptographic algorithm requires comprehensive security evaluation to ensure safe deployment. A notable trend in symmetric cryptanalysis is the adoption of automated methods, which enhance analytical depth and security assessment efficiency [17]. Modern automated cryptanalysis mainly uses mathematical problem solvers to search for distinguishers. Common methods include mixed integer linear programming (MILP) [18], boolean satisfiability (SAT)/satisfiability modulo theories (SMT), and constraint programming (CP) [19]. At the Asiacrypt conference 2016, Xiang et al. [20] proposed a MILP model constructed by inscribing bit-level differentiability transfer properties with inequalities. They automated the search for bit-level differentiability-based integral differentiators using open source MILP solvers. At the Eurocrypt conference 2017, Sasaki et al. [21] proposed a new MILP-based tool for searching impossible differential paths. In 2020, Boura et al. [22] proposed a bit-based MILP modeling method using 8-bit S-boxes and differential diffusion of block ciphers with complex linear layers, which were applied to impossible differential analysis. In 2021, Sadeghi et al. [23] proposed a method to verify the existence of correct plaintext pairs of difference-based distinguisher by using MILP. Despite the progress in MILP-based cryptanalysis, mathematical models for MILP are highly specific and typically require the construction of appropriate mathematical models for specific algorithms and attack types. However, there are still many mathematical models that have not yet been found suitable for solving by MILP methods. Therefore, constructing suitable models for these problems and optimizing the solution using MILP methods are crucial. Improving the existing cryptanalysis results through these means has become one of the current research hotspots in the field of cryptanalysis.

By analyzing the above classical algorithms and security assessment methods, we identify two common limitations. First, the Feistel structure algorithm performs transformation operations on only half of the internal states in each encryption round, resulting in a slower diffusion rate compared to the SPN structure under the same block length. This structural property makes it challenging for the traditional SPN structure to guarantee a sufficient number of active S-boxes in the core round function. Second, algorithms designed with AND, Rotation and XOR (ARX) structures introduce nonlinear properties through logical AND or OR operations. However, traditional methods cannot effectively assess their resistance to differential and linear cryptanalysis, leading to incomplete security guarantees for such algorithms. To address these issues, this paper proposes a logic combination method combining Feistel structure and S-P-S operation, and design a lightweight block cipher algorithm. Moreover, We construct a half-byte automated analysis model based on MILP and employ the Gurobi solver to achieve verifiable security evaluation. The main contributions of this paper are as follows:

1) In order to improve the security of the ciphers, we

TABLE I
NOTATION DESCRIPTION TABLE

Notation	Description
$\mathbb{F}_2$	A finite field containing only two elements 0 and 1
$\mathbb{F}_{2^8}$	Finite fields containing 2^8 elements
$\mathbb{F}_2^l$	The 1-dimensional vector on $\mathbb{F}_2$
$GL(n, \mathbb{K})$	N-rank invertible matrices on domain $\mathbb{K}$
$\oplus$	Exclusive or
$ $	Serialization based on bits or half bytes
SHI^a	Cycle shift a half byte to the left.

propose an optimized logical combination method combining Feistel structure and S-P-S operation, and designs a lightweight block cipher algorithm, called Splight. Compared with the traditional SP structure, the number of active S-boxes can be doubled, which enhances the security of the algorithm. Meanwhile, the proposed combination method balances diffusivity in ensuring state confusion and effectively reduces hardware overhead.

- 2) In order to efficiently and comprehensively evaluate the security of algorithms, this paper proposes an automated evaluation model based on MILP. The model uses MILP to describe the algorithm's cryptographic state based on half-byte and employs the Gurobi automatic search tool for solving. Compared with the traditional cryptanalysis model, it can more comprehensively evaluate whether the algorithm can resist common attacks.
- 3) We introduce three indicators, namely average avalanche, uncertainty and avalanche threshold for full avalanche testing. Through avalanche effect experiments, Splight is compared with the current classical Feistel ciphers SIMON, SPECK and LBLOCK. It can be seen that the diffusion of Splight is better than SIMON and LBLOCK, and the minimum number of rounds to satisfy the avalanche effect is 6. Moreover, Splight has the smallest uncertainty so that the diffusion is the most stable. Therefore, it can be concluded that Splight has good avalanche effect.
- 4) The software tests show that compared with other algorithms, the algorithm proposed in this paper has faster running time and high running efficiency after one thousand, ten thousand, one hundred thousand, and one million times of encryption and decryption. The hardware tests show the round-based hardware structure description and ASIC implementation of Splight in different process library environments. Splight has low power and high throughput hardware characteristics for resource-constrained IoT nodes.

The remainder of this article is organized as follows. We introduce specifics of Splight and design principles in Sections II and Sections III. Section IV analyzes the security of Splight. Section V introduction performance analysis of the software and hardware. Section VI concludes this article.

II. SPECIFICATION OF SPLIGHT

In this section, we introduce the specification of Splight, including encryption, key expansion and decryption algorithms. The symbols used in the following text are listed in Table I.

TABLE II
S-BOX TRUTH TABLE

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	A	D	B	F	9	E	1	0	7	2	5	4	3	6	B	8

x_0	x_4
x_1	x_5
x_2	x_6
x_3	x_7

Fig. 1. Left branch state arrangement diagram.

A. Encryption Algorithm

The plaintext is denoted as $X = b_0b_1b_2, \dots, b_{63}$, where $b_i \in \mathbb{F}_2 (i = 0, 1, \dots, 63)$. Splight uses the Feistel structure, so the plaintext is divided into left and right branches each with 32 bits, i.e. $X = X^l || X^r = b_0b_1 \dots b_{31} || b_{32}b_{33}, \dots, b_{63}$. The left branch is divided into 8 half-bytes, denoted as

$$X^l = b_0b_1b_2b_3 || b_4b_5b_6b_7 || \dots || b_{28}b_{29}b_{30}b_{31} \quad (1)$$

$$\triangleq x_0 || x_1 || \dots || x_7$$

Where $x_i \in \mathbb{F}_2^4 (i = 0, 1, \dots, 7)$. The 8 half-bytes are arranged as a matrix of 4×2 . As the fundamental component of the Splight system architecture, the state arrangement of the left branch is depicted in Fig.1, which defines the basic data format for all subsequent encryption and key expansion operations.

The key of Splight is denoted as K_i and the encryption process is as follows:

$$X_0^l = X^l, X_0^r = X^r \quad (2)$$

$$\begin{aligned} X_i^l &= SHI^2(f(X_{i-1}^l, K_{i-1}) \oplus X_{i-1}^r) \\ X_i^r &= X_{i-1}^l, i = 1, 2, \dots, 32 \end{aligned} \quad (3)$$

$$Y = X_{32}^r || X_{32}^l \quad (4)$$

The f is the core round function of the algorithm, which satisfies

$$f(X_{i-1}^l, K_{i-1}) = S(L \circ S(X_{i-1}^l) \oplus K_{i-1}) \quad (5)$$

where SHI^2 denotes a circular left shift of 8 bits. The S denotes the nonlinear layer of the round function that performs 8 parallel S-box operations on the current state. The procedure is as follows

$$S : \mathbb{F}_2^4 \times \dots \times \mathbb{F}_2^4 \rightarrow \mathbb{F}_2^4 \times \dots \times \mathbb{F}_2^4 \quad (6)$$

$$(x_0, \dots, x_7) \mapsto (S(x_0), \dots, S(x_7)) \quad (7)$$

The truth table for the S-box is shown in Table II.

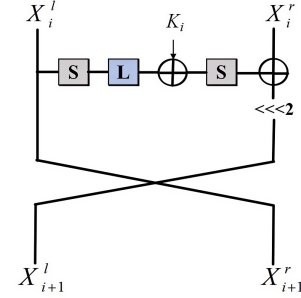


Fig. 2. Round i encryption process.

The L denotes the linear layer of the round function, which applies two parallel column mixing operations to the current state. The detailed procedure is

$$\begin{aligned} L(x_0, x_1, \dots, x_7) = \\ \mathcal{M}(x_0, x_1, x_2, x_3) || \mathcal{M}(x_4, x_5, x_6, x_7) \end{aligned} \quad (8)$$

The column mixing operation is denoted by $\mathcal{M}$, and it is formalized as an 8 bits-based transformation of the state $GL(4, \mathbb{F}_{2^4})$, where M is

Algorithm 1 Splight Encryption Routine.

Input: Plaintext $X = X^l || X^r$, round keys $K_0, K_1, \dots, K_{31}$, $S : \mathbb{F}_2^4 \times \dots \times \mathbb{F}_2^4 \rightarrow \mathbb{F}_2^4 \times \dots \times \mathbb{F}_2^4$, $L(\cdot)$;
Output: Ciphertext Y ;
1: $(X_0^l, X_0^r) \leftarrow (X^l, X^r)$;
2: **for** $i = 1$ to 32 **do**
3: $f(X_{i-1}^l, K_{i-1}) \leftarrow S(L \circ (S(X_{i-1}^l)) \oplus K_{i-1})$;
4: $X_i^l \leftarrow SHI^2(f(X_{i-1}^l, K_{i-1}) \oplus X_{i-1}^r)$;
5: $X_i^r \leftarrow X_{i-1}^l$;
6: **end for**
7: $Y \leftarrow X_{32}^r || X_{32}^l$;
8: **return** Y ;

$$M = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix} \quad (9)$$

Then $\mathcal{M}(x_0, x_1, x_2, x_3) = M \cdot (x_0, x_1, x_2, x_3)^T$. denote $\mathcal{M}$ as a transformation on $\mathbb{F}_2^{16}$, i.e., a bit-based linear transformation whose transformation matrix M' is

$$M' = \begin{pmatrix} M'_1 & M'_2 \\ M'_3 & M'_4 \end{pmatrix} \quad (10)$$

where M'_1 , M'_2 , M'_3 and M'_4 are 8×8 matrices, respectively:

$$M'_1 = \begin{pmatrix} E_{4 \times 4} & A_{4 \times 4} \\ E_{4 \times 4} & O_{4 \times 4} \end{pmatrix}, A_{4 \times 4} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (11)$$

$$M'_2 = \begin{pmatrix} E_{4 \times 4} & E_{4 \times 4} \\ O_{4 \times 4} & O_{4 \times 4} \end{pmatrix}, M'_3 = \begin{pmatrix} O_{4 \times 4} & E_{4 \times 4} \\ E_{4 \times 4} & O_{4 \times 4} \end{pmatrix} \quad (12)$$

$$M'_4 = \begin{pmatrix} E_{4 \times 4} & O_{4 \times 4} \\ E_{4 \times 4} & O_{4 \times 4} \end{pmatrix}$$

During the encryption process of Splight, after S and L transformations, the left-branch state is XORed with the round key. Next, after an S transformation, it undergoes an XOR operation with the right-branch state. After that, an 8-bit cyclic left shift is executed, as described in Algorithm 1. Based on the basic state representation in Fig.1, Fig.2 details the core encryption round process of the Splight system architecture, which is the iterative unit of the entire 32-round encryption system and contains all key operations of the cipher's encryption core.

B. Key Expansion Algorithm

The Splight uses a 128-bit master key and 32-bit round subkeys. The encryption/decryption process requires 32 round subkeys generated by a key expansion algorithm. This algorithm employs a 4-branch generalized Feistel structure, where each branch state is 32 bits and is similar to the plaintext state arrangement. Denote the 128-bit master key as k . The round key generation process for K_i is

$$k = k_0^0 || k_1^0 || k_2^0 || k_3^0 \quad (13)$$

$$k_0^i = f'(k_0^{i-1}) \oplus k_1^{i-1} \oplus (0^{27} || C_{i-1}) \quad (14)$$

$$k_1^i = k_2^{i-1}, k_2^i = k_3^{i-1}, k_3^i = k_0^{i-1}, K_{i-1} = k_0^i \quad (15)$$

Where $i = \{1, 2, \dots, 32\}$, and the f' denotes the core transformation function. This function uses two parallel 4 bits S-box transformations and a cyclic left shift based on half-byte. The S-box is the same as the one used in the encryption algorithm. Let $k_0^{i-1} = k_{0,0}^{i-1} || k_{0,1}^{i-1} || k_{0,2}^{i-1} || k_{0,3}^{i-1} || k_{0,4}^{i-1} || k_{0,5}^{i-1} || k_{0,6}^{i-1} || k_{0,7}^{i-1}$, then

$$f'(k_0^{i-1}) = SHI^1(k_{0,0}^{i-1} || k_{0,1}^{i-1} || k_{0,2}^{i-1} || S(k_{0,3}^{i-1}) || k_{0,4}^{i-1} || k_{0,5}^{i-1} || k_{0,6}^{i-1} || S(k_{0,7}^{i-1})) \quad (16)$$

The C_{i-1} is a 5 bits round constant that is XORed with the least valid 5 bits of the state. The round constant is generated by a counter, and its generation method is

$$C_{i-1} = i \quad (17)$$

As a key component of the Splight system architecture, Fig. 3 details the subkey generation process for one round of the Splight. This process provides 32 round keys for the 32-round iterative encryption core shown in Fig. 2, forming a closed-loop with the encryption process at the system functional level.

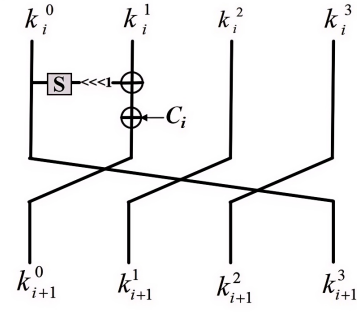


Fig. 3. Subkey generation process for round i.

C. Decryption Algorithm

Considering the characteristics of the Splight algorithm which adopts a Feistel structure, the core round functions of the encryption and decryption algorithms are largely identical. The only difference lies in the shift direction. In encryption, the left branch uses an 8-bit left cyclic shift after being XORed with the right branch. In decryption, it performs an 8-bit right cyclic shift instead. Let Y be the ciphertext and X be the plaintext. The decryption process can be expressed as

$$Y = Y_0^L || Y_0^R \quad (18)$$

$$Y_i^L = f(Y_{i-1}^L, K_{32-i}) \oplus SHI^{-2}(Y_{i-1}^R), \quad (19)$$

$$Y_i^R = Y_{i-1}^L, i = 1, \dots, 32$$

$$X = Y_{32}^R || Y_{32}^L \quad (20)$$

Where the core function f uses the same function as in the encryption algorithm which is not described here.

The decryption algorithm reuses the core encryption module with only minor shift direction adjustments. Figs. 1 to 3 give a layered depiction of Splight's overall system architecture, clearly illustrating all functional units and data interactions by covering the cipher's basic state definition, core encryption round datapath and key expansion module.

III. ALGORITHM DESIGN PRINCIPLES

In this section, we describe in detail the components of round function and the key expansion algorithm, as well as the guidelines for the selection of specific parameters.

A. Round Function

Two common ideas prevail in round function design, which are SP structure and logical instruction combination. Among them, algorithms designed with the logic instruction combination offer higher implementation efficiency and lower cost, making them more adaptable to hardware implementation. However, such algorithms rely on logical operations like AND and OR to introduce nonlinearity, which poses challenges when attempting to evaluate their resistance to differential and linear cryptanalysis through traditional methods. As a result, our understanding of their security remains limited

TABLE III
TRUTH TABLES FOR THE 8 CANDIDATE S-BOXES

$S_1(x)$	A	9	8	F	E	D	4	1	2	3	0	5	7	6	B	C
$S_2(x)$	5	4	3	6	1	0	D	A	F	C	9	E	8	B	2	7
$S_3(x)$	A	D	C	F	9	E	1	0	7	2	5	4	3	6	B	8
$S_4(x)$	8	D	4	7	1	6	3	0	5	2	F	E	9	C	B	A
$S_5(x)$	3	4	9	8	A	F	C	D	E	B	2	1	0	7	6	5
$S_6(x)$	2	A	3	9	0	8	5	E	6	F	7	C	D	4	B	1
$S_7(x)$	2	8	1	9	0	A	7	E	6	D	5	C	F	4	B	3
$S_8(x)$	C	4	B	0	3	A	2	9	1	8	5	F	6	E	7	D

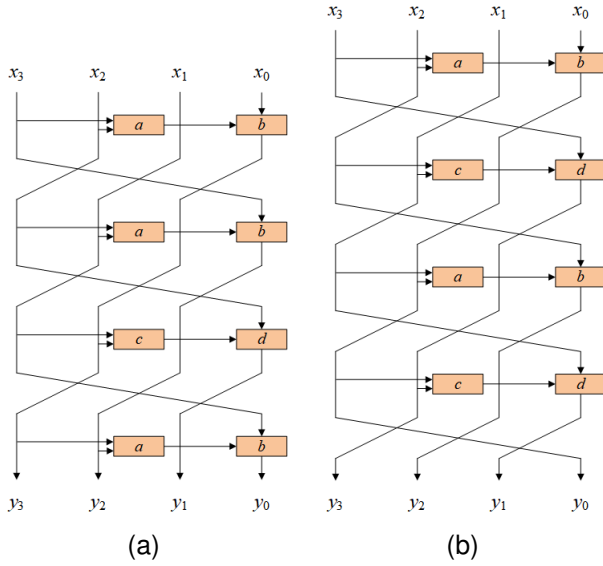


Fig. 4. S-Box Hardware Circuitry. (a) Piccolo algorithm S-Box hardware circuitry (b) uBlock algorithm S-Box hardware circuitry.

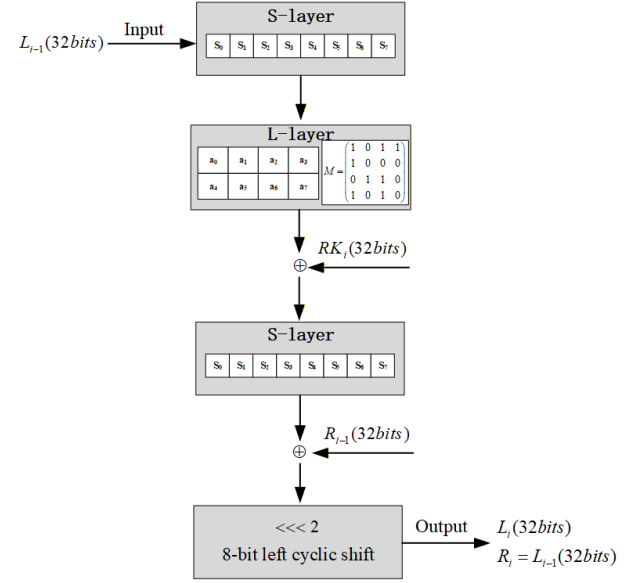


Fig. 5. Detailed architecture of the Splight round function F .

[24]- [26]. Currently, academic research on resistance against differential and linear attacks mainly centers around Simon-like algorithms. Therefore, to facilitate the evaluation of the algorithm's security against typical attacks, Splight will use SP structure to design the core round function.

Splight has a 64-bit block length and its core round function operates on 32-bit states each round. Traditional SP methods may not ensure enough active S-boxes. To solve this, Splight uses an S-P-S structure in its core round function, which doubles the number of active S-boxes and improves security compared to the traditional SP structure. Additionally, to facilitate the utilization of active S-box counting as a metric for evaluating the security strength, it is necessary to introduce randomness between the two S-box layers in the S-P-S structure. Otherwise, the active S-box counts in the two layers cannot be independently assessed. Therefore, we discard the traditional Feistel structure by placing the round key XOR operation between the two S-box layers, ensuring their independence. After the core round function generates a new state and XOR it with the right branch, a word-based left cyclic shift is applied to the resulting 32-bit state. This eliminates potential security risks caused by the cancellation of active S-boxes between the left and right branches.

Fig. 2 illustrates the overall round structure of Splight. A more detailed view of the round function F is provided in

Fig. 5, which shows the two S-box layers, the linear layer L , the placement of the round key XOR, and the 8-bit cyclic shift. The cryptographic properties of the 4-bit S-box and the hardware cost of the linear layer are also annotated.

B. S-box

The selection of the S-box should give priority to minimal hardware implementation costs, optimal differential and linear properties, maximum algebraic degree, and no fixed points. As the only nonlinear component in the round function, the cryptographic properties of the S-box significantly impact the overall security of the algorithm. Balancing security and implementation costs, S-boxes used in cryptographic algorithms typically range between 3 to 8 bits. Generally, larger S-boxes provide stronger security but at higher implementation costs. Since the algorithm employs a Feistel structure with each round operating on a 32-bit state, a 4-bit S-box is chosen. Linear transformations are applied to the state in 4-bit units, ensuring both state confusion and diffusion while facilitating efficient software and hardware implementations.

As the only nonlinear component in the Splight, its resistance to differential and linear attacks is closely related to the cryptographic properties of the S-box, mainly in the differential uniformity and nonlinearity of the S-box [27]. For

a given n bits S-box, the algebraic number $\deg(S)$ of S is defined as

$$\deg(S) = \max_{b \in \mathbb{F}_2^n, b \neq 0} \{\deg(b^T S(x))\} \quad (21)$$

For an n -bit S-box, its algebraic degree is determined by the maximum degree exhibited in the set of all non-zero linear combinations formed from its n output Boolean functions. For any n -bit bijective S-box, this degree is at most $n-1$. Specifically, a 4-bit bijective S-box has a maximum algebraic degree of 3. Thus, a 4 bits S-box with an algebraic degree of 3 is considered optimal.

Considering the security of the S-box and the hardware implementation cost, we investigate the hardware implementation cost of existing lightweight algorithmic S-boxes. The similar 4-branch GFN structure is used to iterate for 4 rounds, and 2 gates are selected per round to construct the lightweight 4-bit S-box. The hardware structure is shown in Fig. 4, where a , b , c , and d represent 4 different 2-in-1 gate circuit structures. Fig. 4(a) shows the S-box of Piccolo when NOR, XOR, NOR, XNOR are used for a , b , c , d . Fig. 4(b) shows the S-box of uBlock when NAND, XOR, NOR, XNOR are used for a , b , c , d , respectively.

Under the SMIC 180nm process library, the NOR and NAND cost 1 gate equivalent (GE) each, and the XOR and XNOR cost 2.67 GE. Thus, by iterating through $a, c \in \{NAND, NOR\}$, $b, d \in \{NOR, XNOR\}$ and considering the positional order of a , b , c , and d in the figure. It is possible to generate a series of 4 bits S-boxes and the hardware implementations all cost $4 \times (1 + 2.67) = 14.68$ GE. Considering the optimal differential and linear properties of S-boxes, eight 4 bits S-boxes are obtained after selection with an algebraic number of 3 and no fixed points. The S-box truth table is shown in Table III.

To evaluate Splight's resistance to integral attacks using the divisibility automation metric [28], we select S-boxes based on minimal effective divisible traces in their propagation characteristics. Using the method proposed by Xiang et al. [29] for calculating S-box divisibility propagation, we analyzed the divisible traces of the eight candidate S-boxes. Among them, S_1 , S_2 , S_4 , S_5 and S_6 each have 46 effective divisible traces, while S_7 and S_8 have 43, and S_3 has 42. Therefore, we selected S_3 as the S-box for the Splight.

C. Linear Layer

The branching states of the Splight algorithm are arranged as a matrix of 4×2 . To enhance implementation efficiency and reduce hardware costs while enabling security evaluation through lower bounds of active S-boxes, we adopt a fixed matrix in $GL(4, \mathbb{F}_{2^4})$ for linear layer transformation. This matrix left-multiplies each column state, effectively obfuscating the four 4-bit nibbles within the same column through optimal algebraic diffusion. In general, maximum distance separable (MDS) matrices are optimally diffusive, and the number of branches of a 4th order MDS matrix is 5. However, MDS matrices require that all of their subformulas are invertible matrices, so any MDS matrix in $GL(4, \mathbb{F}_{2^4})$ must satisfy that

all elements of the matrix are non-zero and that the matrix must contain a non-identity element of $\mathbb{F}_{2^4}$.

State multiplication with non-identity finite field elements requires costly bit-level linear transformations. Thus, the column mixing matrix uses elements that are either identity or zero in the finite field, needing only finite field addition via half - byte XOR operations. This reduces hardware overhead while meeting diffusion requirements. The adopted matrix is

$$M = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix} \quad (22)$$

Note that the first and third elements of the first and fourth rows in the matrix are both set to 1. This structural property enables the elimination of 4 XOR operations in hardware implementations.

Therefore, column mixing for a single column state requires 12 single bit XOR operations. The entire linear layer requires 24, which costs $2.67 \times 24 = 64.08$ GE. After the core round function transformation and XOR operation with the right branch, the left branch undergoes a half-byte based left circular shift to move to the next round's left branch. Since the state is arranged as eight half-bytes at this point, the set of circular shift parameters is $\{0, 1, 2, 3, 4, 5, 6, 7\}$. We use the MILP technique to determine the optimal shift parameters by calculating the minimum number of active S-boxes under different shift parameters. The counts of active S-boxes for different shift parameters are presented in Table IV.

From the differential perspective, when the shift parameters are selected as $\{2, 6\}$, the lower bound of the active S-boxes number is larger compared to other parameters. From the linear perspective, when the shift parameters are selected as $\{2, 3\}$, the lower bound of the active S-boxes number is larger compared to other parameters. Therefore, by taking the intersection of the two sets, the final circular shift parameter is determined to be 2.

D. Key Expansion Layer

Key expansion algorithms can be classified into linear and nonlinear types. Linear designs are vulnerable to invariant subspace and related-key attacks. Consequently, the Splight algorithm employs a nonlinear key expansion design. In the design of key expansion algorithms, three main aspects are taken into account. First, the difference between structure of the key expansion algorithm and the structure of the encryption round function should be maximized. Second, the key expansion algorithm should be a nonlinear expansion algorithm. Finally, the hardware implementation cost of the key expansion algorithm should be minimized.

In order to maximize the difference from the encryption round function structure, the key expansion algorithm of Splight is designed as a 4-branch Feistel structure. Additionally, the core transformation of key expansion uses partial S-boxes, with only two 4-bit S-boxes in each round. The linear layer of the key expansion algorithm implements a left-circular shift operation based on half-bytes, with a shift parameter value of

TABLE IV
LOWER BOUNDS ON THE NUMBER OF DIFFERENTIAL AND LINEAR S-BOXES FOR THE ALGORITHM WITH DIFFERENT SHIFT PARAMETERS

round	differential active S-boxes								linear active S-boxes							
	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	4	4	4	4	4	4	4	4	4	4	5	5	4	4	5	5
4	8	8	8	8	8	8	8	8	8	8	8	8	8	8	7	8
5	10	12	12	12	12	14	12	10	12	12	12	13	12	12	11	12
6	14	16	16	18	20	18	16	14	15	14	16	18	20	17	16	16
7	16	21	24	22	24	24	24	18	16	18	20	24	24	23	22	22
8	20	25	31	26	26	30	31	24	20	24	26	27	26	28	29	27
9	20	29	35	30	30	35	35	26	23	27	31	31	30	30	35	31
10	24	33	41	34	34	39	40	28	24	30	39	38	34	34	40	34
11	26	39	45	40	36	43	44	32	28	33	44	44	36	39	46	39
12	30	43	51	45	40	45	50	36	31	36	49	48	40	42	50	42
13	30	47	55	50	44	49	55	40	32	39	52	53	44	45	53	47
14	34	53	59	53	50	55	61	44	36	42	56	57	50	49	55	52
15	36	57	65	59	52	61	66	49	39	45	60	61	52	55	58	56
16	40	61	69	64	56	67	71	52	40	48	64	67	56	59	63	59

1. Round constants are generated using a counter that does not exceed 32. This approach eliminates the hardware overhead required for storing round constants and reduces the cost of XOR operations involving round constants. As a result, the hardware cost of the core transformation part in each round of key expansion includes only two 4-bit S-boxes and five XOR operations, totaling $2 \times 14.67 + 5 \times 2.67 = 42.69$ GE.

IV. SECURITY ANALYSIS

In this section, we analyze the security of Splight. First, the seven common attacks are analyzed. Second, the MILP model for constructing search differential paths and computing active S-boxes is introduced in the differential analysis. Finally, we perform avalanche testing of Splight and compare it with other algorithms.

A. Differential Analysis

Assessing resistance to differential cryptanalysis is critical in cipher design. Automated analysis techniques provide an efficient security evaluation approach by using public tools to optimize cryptographic properties. For Splight, we used MILP to model its cipher state based on half-bytes and differential propagation, with the aim of minimizing the number of active S-boxes. We solved this model using Gurobi, and the minimum number of active S-boxes per round is shown in Table V. A brief introduction to constructing the MILP model for searching differential paths and counting active S-boxes is presented below.

- 1) **Encoding:** Since the Splight algorithm is a block cipher based on half-bytes, 1-bit variables serve to indicate whether a half-byte is active (1 for active, 0 for inactive). Specifically, the sets of bit variables $(L_0^r, L_1^r, \dots, L_7^r)$ and $(R_0^r, R_1^r, \dots, R_7^r)$ are used to indicate whether the 16 half-bytes of the internal state of the r th round are active or not. Additionally, the sets $(X_0^r, X_1^r, \dots, X_7^r)$, $(Y_0^r, Y_1^r, \dots, Y_7^r)$, and $(Z_0^r, Z_1^r, \dots, Z_7^r)$ are used to indicate whether the half-bytes of the left branch state of the r th round are active after passing through the first

TABLE V
SPLIGHT ALGORITHM DIFFERENTIAL PATHS LOWER BOUND ON THE NUMBER OF ACTIVE S-BOXES

round	1	2	3	4	5	6	7	8
active S-boxes number	0	2	4	8	12	16	24	31
round	9	10	11	12	13	14	15	16
active S-boxes number	35	41	45	51	55	59	65	69

S-box layer, the L-layer, and the second S-box layer, respectively.

- 2) **Modeling:** The MILP model for searching differential paths and quantifying the number of active S-boxes requires modeling the differential propagation of cryptographic components. In the MILP model, the bit variable corresponding to the input half-byte of the S-box is equal to the bit variable corresponding to the output half-byte of the S-box, i.e., $L_i^r = X_i^r$, $Y_i^r = Z_i^r$, $i \in \{0, 1, \dots, 7\}$. Next, we consider the XOR operation in the Splight algorithm's branches. In the Splight, the left branch passes through the round function and is XORed with the right branch. For example, consider the first half-byte XOR in the r -th round branch XOR: the bit variables corresponding to the two input half-bytes and the output half-byte of this XOR operation are denoted as (Z_0^r, R_0^r) and L_2^{r+1} . Then, all the possible differentiation modes of $(Z_0^r, R_0^r, L_2^{r+1})$ are $D = \{(0, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 1), (1, 1, 0)\}$. All the impossible differential modes of are $l = \{(0, 0, 1), (0, 1, 0), (1, 0, 0)\}$. All possible differential modes of $(Z_0^r, R_0^r, L_2^{r+1})$ can be described by the following system of linear inequalities L_{xor} , i.e., all points in the set D satisfy L_{xor} and all points in the set I do not satisfy L_{xor}

$$L_{xor} = \begin{cases} Z_0^r + R_0^r - L_2^{r+1} \geq 0 \\ Z_0^r - R_0^r + L_2^{r+1} \geq 0 \\ -Z_0^r + R_0^r + L_2^{r+1} \geq 0 \end{cases} \quad (23)$$

The linear transformation of Splight uses non-MDS matrices, which cannot be directly modeled using branch

TABLE VI

Rounds	Input Difference ($\Delta L, \Delta R$)	Output Difference ($\Delta L, \Delta R$)
6	(8000 0000, 0000 0000)	(0300 0200, 0010 0010)
7	(8000 0000, 0000 0000)	(0500 0400, 0020 0020)
8	(8000 0000, 0000 0000)	(0A00 0B00, 0040 0020)
9	(8000 0000, 0000 0000)	(0A00 0D00, 0040 0020)

counting. Due to this limitation, it becomes essential to exhaustively enumerate all the input differential modes of the linear transformation and record the values of the bit variables of its inputs and outputs, which produces the point set of possible differential modes. The paper uses a linear closure to obtain the point set of possible differential modes described as L_L . As an example, if we consider the first linear transformation of the r th round function, and the bit variables corresponding to the 8 input and output half-bytes of this linear transformation are $(X_0^r, X_1^r, X_2^r, X_3^r, Y_1^r, Y_2^r, Y_3^r)$, then the construction of L_L is as follows,

$$L_L = \begin{cases} X_0^r - Y_1^r = 0 \\ X_1^r + X_2^r - Y_2^r \geq 0 \\ X_1^r - X_2^r + Y_2^r \geq 0 \\ X_1^r - X_2^r - Y_2^r \leq 0 \\ X_2^r + Y_1^r - Y_3^r \geq 0 \\ X_2^r - Y_1^r + Y_3^r \geq 0 \\ X_2^r - Y_1^r - Y_3^r \leq 0 \\ X_3^r + Y_1^r - Y_3^r \geq 0 \\ X_3^r - Y_1^r + Y_3^r \geq 0 \\ X_3^r - Y_1^r - Y_3^r \leq 0 \end{cases} \quad (24)$$

- 3) **Restrictive condition:** To prevent trivial solutions in the model, the plaintext differential should be non-zero. Therefore, the number of active half-bytes in the plaintext is constrained to be at least 1, i.e.,

$$\sum_{i=0}^7 l_i^0 + R_i^0 \geq 1 \quad (25)$$

- 4) **Objective function:** Since the S-box input bit variables for the r th round of the Splight algorithm are $(L_0^r, L_1^r, \dots, L_7^r)$ and $(Y_0^r, Y_1^r, \dots, Y_7^r)$. The objective function of the MILP model for searching the number of active S-boxes in R -round differential paths is

$$\min \sum_{r=0}^{R-1} \sum_{i=0}^7 L_i^r + Y_i^r \quad (26)$$

As shown in Table V, the minimum number of active S-boxes in 9 rounds is 35. Based on the characteristics of S-boxes and differential probability calculations, with a maximum differential probability of 2^{-2} per S-box, the maximum differential propagation probability for 9-round differential paths in the Splight algorithm is $DP_2 \leq 2^{-35 \times 2} = 2^{-70} < 2^{-64}$. This demonstrates that there are no exploitable 9-round differential paths, which in turn proves that the full-round Splight algorithm is resistant to differential attacks.

The differential paths presented in Table IX demonstrate that the theoretical lower bounds established by MILP analysis are

TABLE VII
SPLIGHT ALGORITHM LINEARLY APPROXIMATES A LOWER BOUND ON THE NUMBER OF ACTIVE S-BOXES

round	1	2	3	4	5	6	7	8
active S-boxes number	0	2	5	8	12	16	20	26
round	9	10	11	12	13	14	15	16
active S-boxes number	31	39	44	49	52	56	60	64

practically attainable. For each round count (6-9), we provide a concrete differential path starting from the input difference $(\Delta L, \Delta R) = (8000\ 0000, 0000\ 0000)$ and reaching the exact number of active S-boxes specified in Table VII: 16 active S-boxes for 6 rounds, 24 for 7 rounds, 31 for 8 rounds, and 35 for 9 rounds. These paths were discovered using the same MILP model employed for bound calculation, confirming the tightness of our theoretical analysis. Notably, the corresponding differential probabilities, calculated as $2^{-2 \times \text{active S-boxes}}$, yield 2^{-32} , 2^{-48} , 2^{-62} , and 2^{-70} respectively. Even the 8-round path already provides a probability bound (2^{-62}) below the 2^{-64} security threshold for a 64-bit block cipher, while the 9-round path further strengthens this margin to 2^{-70} . These instances validate that Splight offers substantial security reserves against differential cryptanalysis.

B. Linear Analysis

Attackers employ plaintext-ciphertext pairs and linear approximations for key recovery, and they require effective multi-round linear approximations. Resistance to linear attacks is assessed by evaluating active S-boxes using MILP, analogous to the differential analysis approach. Specifically, we model linear mask propagation based on half-bytes, set minimizing active S-boxes as the objective, and solve using Gurobi. Since model details are similar to the differential analysis model, they are not repeated here. Table VII lists the minimum number of active S-boxes for linear approximations from 1 to 16 rounds.

TABLE VIII
LINEAR APPROXIMATION PATHS WITH CORRELATION BOUNDS

Rounds	Input Mask ($\Gamma L, \Gamma R$)	Output Mask ($\Gamma L, \Gamma R$)
7	(0100 0000, 0000 0000)	(0040 0020, 0001 0000)
8	(0100 0000, 0000 0000)	(0060 0030, 0002 0001)
9	(0100 0000, 0000 0000)	(0080 0040, 0002 0001)
10	(0100 0000, 0000 0000)	(0080 0040, 0002 0001)

As shown in Table VII, the minimum number of active S-boxes in a 10-round linear approximation of Splight is 39. Given that the squared effective linear correlation coefficient of a single S-box is bounded by 2^{-2} . Therefore the square of the correlation coefficient for 10 rounds of linear approximation is $LC_{10} \leq 2^{-2 \times 39} = 2^{-78} < 2^{-64}$. This demonstrates that no exploitable 10-round linear approximations exist, which confirms that the full-round Splight is resistant to linear attacks.

The linear approximation paths detailed in Table VIII offer empirical verification of the theoretical lower bounds derived for linear cryptanalysis. Starting from the input mask

TABLE IX
SPLIGHT ENCRYPTION ALGORITHM FULL DIFFUSION ROUNDS

half-byte	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7
diffusion rounds	4	5	4	5	4	5	4	5
half-byte	x_8	x_9	x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	x_{15}
diffusion rounds	5	6	5	6	5	6	5	6

$(\Gamma L, \Gamma R) = (0100\ 0000, 0000\ 0000)$, these paths achieve active S-box counts of 20, 26, 31, and 39 for 7, 8, 9, and 10 rounds respectively, matching exactly the theoretical minima from Table ?? . The squared correlation bounds for these paths, computed as $2^{-2 \times \text{active S-boxes}}$, are 2^{-40} , 2^{-52} , 2^{-62} , and 2^{-78} in sequence. The 9-round approximation already yields a squared correlation bound (2^{-62}) below the 2^{-64} security threshold, and the 10-round path extends this safety margin to 2^{-78} . These results not only confirm the tightness of the MILP-derived linear bounds but also substantiate that even optimal linear approximations remain cryptographically negligible for the full 32-round Splight algorithm.

C. Impossible Differential Analysis

To search for impossible differential paths in round r of Splight, we conducted an exhaustive test on all combinations of 16 input and 16 output unit vectors, totaling $16 \times 16 = 256$ models to solve. In the case that any model is unsolvable, it confirms an impossible differential path for round r and triggers the search for the next round. If all 256 models are solvable, it indicates that no impossible differential paths exist for that r round. Experimental results demonstrate that the longest impossible differential path for Splight has a length of 11 rounds, indicating that the full - round algorithm can resist impossible differential attacks.

D. Zero-Correlation Linear Analysis

As the differential and linear evaluation models for the Splight algorithm are similar, the zero-correlation linear and impossible differential evaluation models share similar characteristics. Thus, the evaluation method for zero-correlation linear can be based on the above impossible differential analysis. Consequently, experimental results show that the longest zero-correlation linear trace for the Splight algorithm is 10 rounds. Considering that the algorithm has 32 rounds, the full-round Splight algorithm can effectively resist zero-correlation linear attacks.

E. Integral Analysis

For the integral cryptanalysis of Splight, this section utilizes the technique proposed by Sun et al. [30] to search for the longest-round integral distinguisher of the algorithm. Experimental results indicate that Splight exhibits a 9-round integral distinguisher but there is no 10-round integral distinguisher. In the following, we present an example of a 9-round integral distinguisher:

$$(caaaAAA, AAAA) \xrightarrow{\text{Nine-round}} (BUBB, UBUB) \quad (27)$$

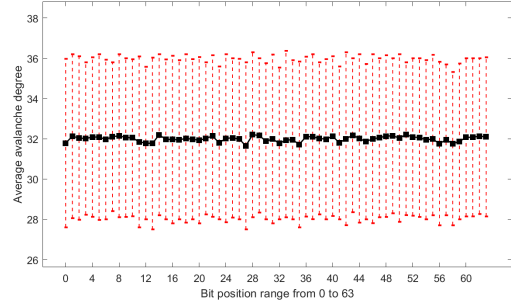


Fig. 6. Average avalanche degree of Splight.

Where “ A ” and “ a ” denote that the input half-byte and bit are active, “ C ” and “ c ” denote that the input half-byte and bit are constant, “ B ” and “ b ” denote that the output half-byte and bit are balanced, and “ U ” and “ u ” denote that the output half-byte and bit are unbalanced, respectively.

Since the total number of rounds of Splight is significantly greater than the number of rounds for which integral distinguishers can be constructed, the full-round Splight can resist integral attacks.

F. Diffusivity Analysis

The Splight algorithm is based on half-bytes. Its nonlinear layer uses 4 - bit S - boxes. Since the four input bits of the S-box affect all four output bits, in order to test the diffusion property of the Splight algorithm, this section tests the full diffusion of the encryption algorithm in half-byte units. The initial 64-bit state X is divided into 16 4-bit words, which are represented as $X = x_0 || x_1 || \dots || x_{14} || x_{15}$. To test the r -round diffusivity of $x_i (i \in 0, 1, \dots, 15)$, let

$$x_i = \begin{cases} 1, & j = i \\ 0, & j \neq i \end{cases} \quad (28)$$

Where $j \in \{0, 1, 2, \dots, 15\}$. We use the MILP technique to describe the propagation in the r -round encryption for the 16 half-byte initial state and makes the maximum value in the r -round state as the objective function. If the solution of the model corresponding to the $r-1$ round is less than 16 and the solution of the r -round model is equal to 16, then it means that the number of fully diffused rounds x_i is r . The diffusivity of the initial state of 16 half-bytes has been tested according to the above method, and the results of the experimental tests are shown in Table IX. From the data in the table, it can be seen that the number of rounds to reach full diffusion for states in different locations is slightly different, with a minimum of 4 and a maximum of 6. Therefore, the number of rounds for full diffusion of states for Splight is 6.

G. Avalanche Test

In cryptography, the avalanche effect means small input changes cause significant output changes. In a well-designed block cipher, minor key or plaintext changes must make ciphertext changes unpredictable. Without enough avalanche properties, a cryptographic function is considered to have poor

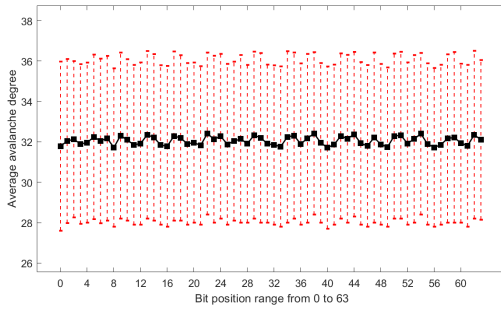


Fig. 7. Average avalanche degree of SIMON.

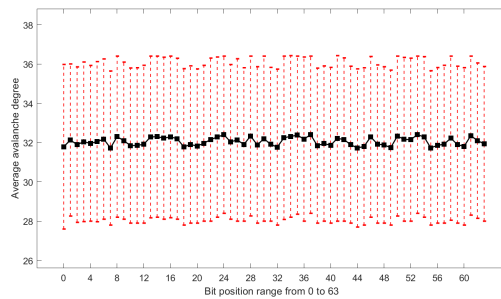


Fig. 8. Average avalanche degree of SPECK.

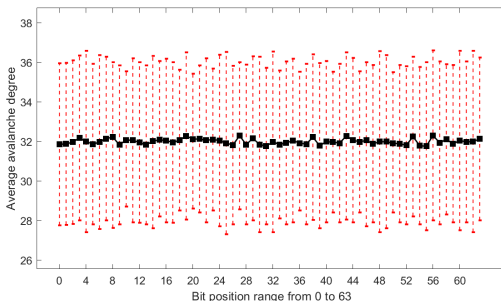


Fig. 9. Average avalanche degree of LBLOCK.

randomization, allowing cryptanalysts to deduce input details from outputs, potentially leading to algorithm cracks. Thus, the avalanche effect is of crucial importance to cryptographic algorithm designers. This section involves the computation of two metrics: the average avalanche degree per bit and the uncertainty degree, which jointly characterize data fluctuation ranges. Expanded data fluctuations accompanied by increased uncertainty indicate weakened diffusion stability within the algorithm, while reduced fluctuation magnitudes with lower uncertainty levels correspond to enhanced stability. In statistical mathematics, variance serves as a conventional measure for quantifying such uncertainty.

The avalanche test procedure for Splight involves the following steps. Initially, a fixed set of keys is chosen along with a selected set of plaintexts. The encryption process yields an initial ciphertext C . Next, the first bit of the plaintext is flipped while keeping the remaining bits the same, and the modified plaintext is encrypted with the same key to produce ciphertext

TABLE X
COMPARISON OF AVALANCHE EFFECT DATA OF VARIOUS CIPHERS

Algorithm	Block size	Key size	Average avalanche	Average uncertainty	Avalanche threshold
SIMON [13]	64	128	31.98101	3.98951	12
SPECK [13]	64	128	32.01457	4.01225	8
LBLOCK [12]	64	80	31.98143	4.01421	6
Splight	64	128	31.98556	3.98614	6

C_0 . The avalanche effect for the first bit of the plaintext is then calculated by determining $Humming(C \oplus C_0)$. This process is subsequently repeated for every bit in the plaintext P to measure the avalanche degree for each bit. Furthermore, the entire procedure is carried out 1000 times, with the plaintext P being randomized in each iteration while the key remains the initially fixed key.

Based on these experiments, we gathered 1000 sets of avalanche test data. For every bit position, we determined the mean value to represent Splight's average avalanche effect per bit, while the variance served as a measure of the uncertainty for each bit. The results are presented in Fig. 6. The horizontal axis displays the Splight plaintext bits arranged from high to low, with the vertical axis showing the average avalanche level per bit. To indicate data uncertainty, we employed error bars where each bar is centered on the calculated mean value and extends equally in both directions to represent the measurement uncertainty range.

The avalanche effects of SIMON, SPECK, and LBLOCK are tested using the same method, as depicted in Fig. 7, Fig. 8, and Fig. 9. From these figures, one can clearly observe the average avalanche degree of each bit for the aforementioned algorithms and the fluctuation range of the data throughout the entire avalanche process. The average avalanche and uncertainty values presented in Table X indicate that SIMON exhibits an average avalanche of less than 32 and an uncertainty of less than 4. In contrast, SPECK has an average avalanche of more than 32 and an uncertainty of more than 4. LBLOCK has an average avalanche of less than 32 and an uncertainty greater than 4. Splight has an average avalanche closest to 32 and an uncertainty of 3.98614. The last column of Table X corresponds to the critical number of rounds required to achieve the avalanche effect. A lower critical number of rounds implies a faster data spreading rate during the encryption process of the algorithm. The minimum number of rounds for Splight to satisfy the avalanche effect is 6. Therefore, by considering the above mentioned metrics, it can be concluded that Splight demonstrates a good avalanche effect.

V. PERFORMANCE ANALYSIS

In this section, we analyze the performance of Splight from both hardware and software aspects. Specially, the performance of Splight is evaluated by comparing it with some existing lightweight block ciphers.

A. Software Analysis

The smallest operational unit of the Splight algorithm is 4 bits. For software implementation, we use a lookup table

TABLE XI
SOFTWARE PERFORMANCE COMPARISON OF LIGHTWEIGHT BLOCK CIPHERS

Cipher	Block/Key (bits)	Software Performance	
		Cycles/byte	Mbps
Splight	64/128	154.5	224.2
PRESENT	64/128	192.3	175.6
GIFT	64/128	168.7	189.8
LED	64/128	205.4	155.9
PRINCE	64/128	176.9	180.9
SIMON	64/128	142.1	205.3
SKINNY	64/128	161.8	197.9
CLEFIA	128/128	231.5	110.6

TABLE XII
CRYPTOGRAPHIC ALGORITHM EXECUTION TIME

	10^3	10^4	10^5	10^6
Present [11]	89	240	4396	18695
Led [31]	67	735	1510	17391
Splight	3	140	1261	9856
LBlock [12]	9	344	1394	13345
AES [10]	256	1578	6589	26284

method based on C language to implement the algorithm. This method ensures both the efficiency and applicability of the implementation strategy. State storage, S-box implementations, linear layer implementations, and key extensions can be achieved by constructing lookup tables. For state storage, the algorithm's state length is 64 bits. It is stored using four 16-bit unsigned integer variables. Also, two 16-bit unsigned integer variables serve as intermediate variables for the round function. For the S-box transformation after the round key addition in the algorithm, we construct a uint8_t array of length 256 as a lookup table using two S-box transformations simultaneously. For the S-box transformation before the linear layer in the round function, we combine it with the linear layer implementation to construct the corresponding composite lookup table. In the linear layer implementation, we construct two lookup tables T_1 and T_2 as uint16_t arrays of length 256. This allows the S-box layer and linear layer transformations before the round key addition to be completed in one go, with just two table lookups and one XOR operation. For the key expansion, we use four 32-bit unsigned integers to store the expanded key and two more as intermediate variables for the key expansion function. The key expansion function has 32 rounds. The S-box transformation in it uses the same lookup table operation as in the round function, and each round outputs a 32-bit round key.

TABLE XIII
ASIC REALIZES STANDARD GATE AREA

Standard gate circuit	Equivalent door numbers(GE)
NAND	1
XOR	2.67
NOR	1
XNOR	2.67
MUX	2.67
D Flip Flop	5.33
Scan Flip Flop	6.67

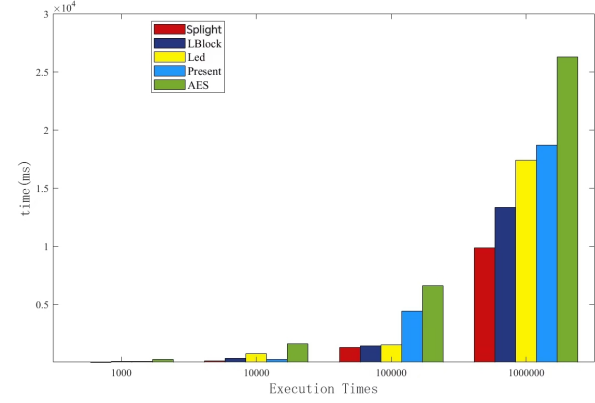


Fig. 10. Splight and other cryptographic algorithms execution time comparison.

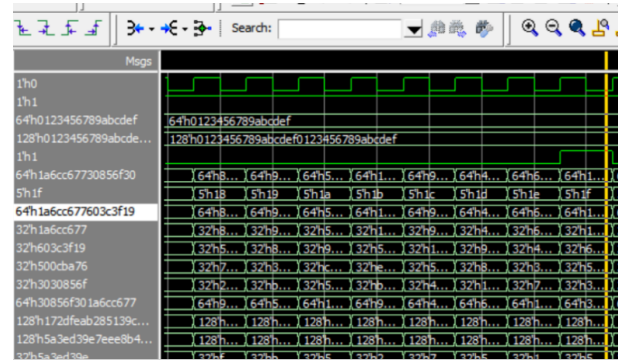


Fig. 11. ModelSim simulation result.

To evaluate the software efficiency of Splight and position it within the landscape of lightweight cryptography, we conducted comprehensive performance tests. Under the environment of Intel Core i5-10200H, 2.4GHz, 16GB RAM, 64-bit Window 11 system, and C language compilation, we randomly selected plaintext and seed keys, repeated the experiments several times, and took the averages. The speed metrics were cycles per byte (cycles/byte) and megabits per second (Mbps). The experimental results show that the encryption and decryption rates of Splight are 224.16Mbps and 386.57Mbps, respectively. The difference between the encryption and decryption rates arises from software-level

TABLE XIV
EQUIVALENT NUMBER OF GATES BY ROUND REALIZATION SCHEME

Modular component		Equivalent gates number
State		6.67×64=426.88
Key		6.67×128=853.76
Logic Control		43.67
Round function	Sbox	14.67×16=234.72
	MDS	2.67×12×2=64.08
	left-right branch xor	2.67×32=85.4
	key xor	2.67×32=85.4
	shift	0
Key expansion	Sbox	14.67×2=29.34
	branch xor	2.67×32=85.4
	round constant xor	2.67×4=10.68
Total		1918

TABLE XV
HARDWARE PERFORMANCE COMPARISON OF LIGHTWEIGHT BLOCK CIPHERS UNDER THREE PROCESS LIBRARIES

	NangateOpen45nm				TSMC90nm				SMIC180nm			
	Area (um)	Freq (MHz)	Power (mW)	Tput (Gbps)	Area (um)	Freq (MHz)	Power (mW)	Tput (Gbps)	Area (um)	Freq (MHz)	Power (mW)	Tput (Gbps)
Splight	2984	484.6	0.7	2.97	10467	234.3	1.56	1.33	39259	571.4	3.44	1.14
PRESENT	3152	549.1	0.9	1.57	11234	283.7	1.89	0.64	37481	263.2	3.43	0.39
PRINCE	3677	413.5	1.6	2.89	10536	165.5	1.64	1.12	42736	149.6	3.87	0.82
AES	25981	296.3	3.4	3.62	51258	136.2	12.5	0.97	60230	89.1	15.6	0.74
LED	2816	336.4	0.5	0.84	11839	123.5	1.92	1.51	31210	105.2	3.09	1.34
GIFT	2873	436.8	0.9	1.24	11347	203.6	1.78	0.91	38842	416.7	3.76	0.81
SIMON	2694	493.2	0.8	2.48	10368	312.8	1.67	1.19	37105	492.9	3.98	1.03
SKINNY	3124	463.4	1.8	2.03	11728	228.6	1.68	1.12	39723	452.5	3.21	0.58
CLEFIA	4723	334.2	2.7	1.17	22648	192.4	2.96	0.82	49317	304.9	5.38	0.35

optimizations, specifically the caching of the precomputed key schedule and the efficient reuse of composite lookup tables during decryption. These optimizations leverage the symmetry of the Feistel structure, leading to improved memory access patterns and cache utilization for the decryption process in our test environment.

To directly compare software performance with other lightweight cryptographic algorithms, Table XI summarizes the execution efficiency of Splight alongside CLEFIA [31], GIFT [32], LED [33], PRESENT, PRINCE, SIMON, and SKINNY. This comparison demonstrates that Splight achieves competitive software efficiency at 154.5 cycles/byte and 224.2 Mbps throughput. Specifically, Splight outperforms PRESENT, GIFT, LED, PRINCE, and CLEFIA in terms of cycles per byte while demonstrating comparable performance to SKINNY. The software performance advantage of Splight can be attributed to its design optimizations for efficient implementation. The use of 4-bit S-boxes enables compact lookup tables that fit well in processor cache, reducing memory access latency. The hybrid lookup table approach that combines the S-box and linear layer operations minimizes the number of table lookups per round. Additionally, the semi-byte-oriented operations align with the data processing characteristics of general-purpose processors. These design choices collectively contribute to Splight's software performance while maintaining the security benefits of the S-P-S structure.

Moreover, we conducted encryption and decryption tests on Splight, PRESENT-64/128, LBlock, LED-64/128, and AES-64/128 algorithms. In ECB mode, the plaintext length for all tested algorithms was 64 bits. We ran each algorithm 1,000, 10,000, 100,000, and 1,000,000 times for encryptions and decryptions, with the results shown in Table XII. In order to visualize rate comparisons, we plotted a histogram in Fig. 10, where the x-axis indicates the number of runs and the y-axis shows the runtime in milliseconds. The experimental results show that decrypting the algorithm 1,000,000 times took 9856 ms for Splight and 13345 ms for LBlock. In conclusion, our proposed Splight demonstrates high software runtime efficiency.

Compared with other lightweight block ciphers, including the standardized performance metrics in Table XI and the actual runtime tests in Table XII and Fig.10, the results collectively demonstrate that Splight exhibits superior software performance, making it well-suited for resource-constrained

embedded systems.

B. Hardware Analysis

We implement Splight in a round-based way using the Verilog hardware description language. The simulation is performed in ModelSim-64 10.5 using TestBench. During simulation, the TestBench is used for two main purposes. First, it generates test excitations for the design to be tested. Second, it checks if the outputs of the design match the expected results to verify the design function. The TestBench of the simulation software ModelSim-64 10.5 inputs test signals. The plaintext is set to 01 23 45 67 89 AB CD EF, and the initial key is set to 01 23 45 67 89 AB CD EF 01 23 45 67 89 AB CD EF. As shown in Fig. 11, when the encryption completion signal o_done is 1, o_data is the encryption result. The ciphertext is 1a6cc67730856f30, which matches the expected result.

In this paper, we use Verilog HDL hardware description language to model and describe the hardware structure of Splight. Synopsys DesignCompiler 2015 is used to synthesize the hardware code, and the target process library is SMIC 180nm. The input interfaces are i_clk, i_enable, i_plaintext, and i_key. The output interfaces are o_done, o_data. Regarding power consumption, we suggest the compiler should keep a hierarchical structure and use a 100KHz clock frequency, common in RFID applications. The round-based circuit includes the state matrix register, key register, control module, round function module, and various modules in the key expansion function. It stores data in a 64-bit state matrix register and keys in a 128-bit key register, updating them after each round. Under the SMIC 180nm process, hardware costs are: 1 GE for NAND and NOR, 2.67 GE for XNOR, XOR, and MUX selectors, 5.33 GE for D Flip Flop registers, and 6.67 GE for Scan Flip Flop registers. Table XIII lists these metrics, which help estimate hardware costs of Splight.

In NangateOpen45nm, TSMC90nm, and SMIC180nm process libraries, we compare the implementations of PRESENT-64, PRINCE-64, AES, Led [28], and additionally CLEFIA, GIFT, SIMON, and SKINNY with Splight. We test their hardware occupied area, frequency, power, and throughput across different sizes and libraries to evaluate Splight's hardware performance in Table XV. For area, Splight is only 2984 μm^2 at the Nangate 45nm process, which is 5.3% and 18.8% less than PRESENT and PRINCE, and also demonstrates competitive area efficiency compared to GIFT, SIMON, and SKINNY.

This is because the full-state iterative structure breaks the diffusion bottleneck of the traditional Feistel half-state update, and realizes deep logic compression with the 4-bit S-box and parallel replacement layer. For power consumption, Splight uses a round-based hardware implementation architecture. It consumes 3.44 mW in the SMIC 180nm process, 11.1% lower than PRINCE and significantly lower than CLEFIA. In the TSMC 90nm process, it consumes 1.56 mW, 87.5% lower than AES. For throughput, the 1.14 Gbps throughput driven by 571.4 MHz high-frequency at SMIC180nm is 39% higher than that of PRINCE, and outperforms PRESENT, GIFT, and SKINNY, which is attributed to the single-cycle completion of the S-P-S replacement layer and the parallelization of the hardware design.

VI. CONCLUSION

In this paper, we have proposed an optimized logic combination method combining Feistel structure and S-P-S operation, and designed a lightweight block cipher algorithm called Splight. Splight overcomes the drawback of the existing Feistel structure that transforms only half of the internal state in each round. Second, a half-byte automated analysis model based on MILP is proposed and combined with the Gurobi tool to achieve a verifiable evaluation of the algorithm's security. In addition, we perform a security analysis on Splight to illustrate that the algorithm has high security performance. Finally, we perform software and hardware of Splight and current classical ciphers based on Feistel structure. The results show that Splight is suitable for low-resource and low-power embedded devices. Our future work will include an investigation into side-channel attack countermeasures and the exploration of flexible hardware structures to enable adaptive security-performance trade-offs for diverse application requirements.

REFERENCES

- [1] Q. Lai, J. Tian, W. Wang, and X. Hu, "Spatial-temporal attention graph convolution network on edge cloud for traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4565–4576, Apr. 2023, doi: 10.1109/TITS.2022.3185503.
- [2] B. D. Manh, C.-H. Nguyen, D. T. Hoang, D. N. Nguyen, M. Zeng, and Q.-V. Pham, "Privacy-preserving cyberattack detection in blockchain-based IoT systems using AI and homomorphic encryption," *IEEE Internet Things J.*, doi: 10.1109/JIOT.2025.3535792.
- [3] K. Zheng, Z. Zhou, J. Liu, and B. Yu, "Secure fine-grained multi-keyword ciphertext search supporting cloud-edge-end collaboration in IoT," *Chin. J. Electron.*, vol. 34, no. 1, pp. 266–281, Jan. 2025, doi: 10.23919/cje.2023.00.244.
- [4] S. Ullah et al., "Homomorphic encryption applications for IoT and lightweight environments: A review," *IEEE Internet Things J.*, vol. 12, no. 2, pp. 1222–1246, Jan. 2025, doi: 10.1109/JIOT.2024.3472029.
- [5] X. Chai, Q. Xiong, J. Yu, Z. Gan, Y. Ma, and Y. Zhang, "SMPCS: Self-managed privacy control and sharing scheme based on thumbnail-preserving and ECDH for social networks," *IEEE Internet Things J.*, 2024.
- [6] C. Hu, J. Liu, H. Xia, S. Deng, and J. Yu, "A lightweight mutual privacy preserving k-means clustering in industrial IoT," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 2138–2152, Mar.–Apr. 2024.
- [7] C. Hu, Z. Liu, R. Li, P. Hu, T. Xiang, and M. Han, "Smart contract assisted privacy-preserving data aggregation and management scheme for smart grid," *IEEE Trans. Depend. Secure Comput.*, vol. 21, no. 4, pp. 2145–2161, Jul.–Aug. 2024, doi: 10.1109/TDSC.2023.3300749.
- [8] S. Tatarka, M. Janovec, and J. Papán, "IoT security threats and solutions," in *Proc. Int. Conf. Emerg. eLearn. Technol. Appl. (IC-ETA)*, Stary Smokovec, Slovakia, 2024, pp. 605–610, doi: 10.1109/IC-ETA63795.2024.10850844.
- [9] L. Dalmasso, F. Bruguier, P. Benoit, and L. Torres, "Evaluation of SPN-based lightweight crypto-ciphers," *IEEE Access*, vol. 7, pp. 10559–10567, 2019.
- [10] V. Rijmen and J. Daemen, "Advanced encryption standard," *Proc. Fed. Inf. Process. Stand. Publ.*, Nat. Inst. Stand. Technol., vol. 19, p. 22, 2001.
- [11] A. Bogdanov, L. R. Knudsen, G. Leander et al., "PRESENT: An ultra-lightweight block cipher," in *Proc. Int. Workshop Cryptogr. Hardw. Embedded Syst. (CHES)*, Berlin, Germany, 2007, pp. 450–466.
- [12] W. Wu and L. Zhang, "LBlock: A lightweight block cipher," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, Springer, 2011, pp. 327–344.
- [13] R. Beaulieu, D. Shors, J. Smith et al., "The SIMON and SPECK families of lightweight block ciphers," in *Proc. ACM/EDAC/IEEE Des. Autom. Conf. (DAC)*, 2015, pp. 1–6.
- [14] J. Borghoff, A. Canteaut, T. Güneysu et al., "PRINCE – A low-latency block cipher for pervasive computing applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur. (ASIACRYPT)*, Springer, 2012, pp. 208–225.
- [15] W. Zhang, Z. Bao, D. Lin et al., "RECTANGLE: A bit-slice lightweight block cipher suitable for multiple platforms," *Sci. China Inf. Sci.*, vol. 58, pp. 1–15, 2015.
- [16] C. Beierle, J. Jean, S. Kölbl et al., "The SKINNY family of block ciphers and its low-latency variant MANTIS," in *Proc. Annu. Int. Cryptol. Conf. (CRYPTO)*, Springer, 2016, pp. 123–153.
- [17] S. P. Wang, B. Hu, J. Guan et al., "MILP-aided method of searching division property using three subsets and applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Cham, Switzerland: Springer, 2019, pp. 398–427.
- [18] S. W. Sun, L. Hu, P. Wang et al., "Automatic security evaluation and (related-key) differential characteristic search: Application to SIMON, PRESENT, LBlock, DES(L) and other bit-oriented block ciphers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Berlin, Germany: Springer, 2014, pp. 158–178.
- [19] S. Andrei, G. Grigoras, M. Rinard et al., "A hierarchy of tractable subclasses for SAT and counting SAT problems," in *Proc. 11th Int. Symp. Symbol. Numer. Algorithms Sci. Comput.*, 2009, pp. 61–68.
- [20] Z. Xiang, W. Zhang, Z. Bao, and D. Lin, "Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Berlin, Germany: Springer, 2016, pp. 648–678.
- [21] Y. Sasaki and Y. Todo, "New impossible differential search tool from design and cryptanalysis aspects," in *Proc. Annu. Int. Conf. Theory Appl. Cryptol. Techn. (EUROCRYPT)*, Cham, Switzerland: Springer, 2017, pp. 185–215.
- [22] C. Boura and D. Coggia, "Efficient MILP modelings for Sboxes and linear layers of SPN ciphers," *IACR Trans. Symmetric Cryptol.*, pp. 327–361, 2020.
- [23] S. Sadeghi, V. Rijmen, and N. Bagheri, "Proposing an MILP-based method for the experimental verification of difference-based trails: Application to SPECK, SIMECK," *Des. Codes Cryptogr.*, vol. 89, no. 9, pp. 2113–2155, 2021.
- [24] S. Kölbl, G. Leander, and T. Tiessen, "Observations on the SIMON block cipher family," in *Proc. Annu. Int. Cryptol. Conf. (CRYPTO)*, Berlin, Germany: Springer, 2015, pp. 161–185.
- [25] S. Kölbl and A. Roy, "A brief comparison of Simon and Simeck," in *Proc. Int. Workshop Lightweight Cryptogr. Secur. Privacy*, Cham, Switzerland: Springer, 2016, pp. 69–88.
- [26] G. Leurent, C. Pernot, and A. Schrottenloher, "Clustering effect in Simon and Simeck," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Cham, Switzerland: Springer, 2021, pp. 272–302.
- [27] G. Leander and A. Poschmann, "On the classification of 4 bit S-boxes," in *Proc. Int. Workshop Arithmetic Finite Fields*, Berlin, Germany: Springer, 2007, pp. 159–176.
- [28] Y. Todo and M. Morii, "Bit-based division property and application to Simon family," in *Proc. Int. Conf. Fast Softw. Encrypt.*, Berlin, Germany: Springer, 2016, pp. 357–377.
- [29] Z. Xiang, W. Zhang, Z. Bao, and D. Lin, "Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Berlin, Germany: Springer, 2016, pp. 648–678.
- [30] L. Sun, W. Wang, and M. Q. Wang, "MILP-aided bit-based division property for primitives with non-bit-permutation linear layers," *IET Inf. Secur.*, vol. 14, no. 1, pp. 12–20, 2020.
- [31] Shirai T, Shibutani K, Akishita T, et al. The 128-bit blockcipher CLEFIA (extended abstract)[C].Fast Software Encryption, 14th International Workshop, FSE 2007, Luxembourg, Luxembourg, March 26–28, 2007, Revised Selected Papers. Springer, 2007: 181–195.

- [32] Banik S, Pandey S K, Peyrin T, et al. GIFT: A small present - towards reaching the limit of lightweight encryption[C]. Cryptographic Hardware and Embedded Systems - CHES 2017 - 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings. Springer, 2017:321-345.
- [33] J. Guo, T. Peyrin, A. Poschmann et al., "The LED block cipher," in *Proc. Int. Workshop Cryptogr. Hardw. Embedded Syst. (CHES)*, Berlin, Germany: Springer, 2011, pp. 326–341.



Min Zhang was born in Shanxi, China, in 1994. She is currently pursuing the Ph.D. degree with the Department of Information Security, Naval University of Engineering, Wuhan, China. Her interests include deep learning, Internet of vehicles and privacy protection in crowdsensing.



Qian Zhou was born in Shaanxi, China, in 1999. She is currently pursuing the Ph.D. degree with the Department of Information Security, Naval University of Engineering, China. Her main research interests include the underwater acoustic sensor network security, cryptographic algorithm and its application.



Xiaobing Liu was born in Hunan, China, in 1989. He is currently pursuing the Ph.D. degree with the Department of Information Security, Naval University of Engineering, Wuhan, China. His interests include chaos applications, video encryption and symmetric ciphers.



Qing Ye was born in Hubei, China, in 1978. He received his BS degree in Automation Engineering from Naval Electronics Engineering College, and his MS degree in Vehicle Operation Engineering from Navy Submarine Academy, and his PhD degree in Systems Engineering from Naval University of Engineering, Wuhan, China, in 2008. He is currently a Professor with the Department of Information Security, Naval University of Engineering. His research interests include underwater acoustic sensor network security, cryptographic algorithm and its application,

uncertainty reasoning and decision.



Chengzhe Lai (Member, IEEE) received the B.S. degree in information security from the Xi'an University of Posts and Telecommunications, Xi'an, China, in 2008, and the Ph.D. degree from Xidian University, Xi'an, in 2014. He was a visiting Ph.D. student with the Broadband Communications Research Group, University of Waterloo, Waterloo, ON, Canada, from 2012 to 2014. He is currently a Full Professor with the Xi'an University of Posts and Telecommunications, and is also with the National Engineering Research Center for Secured Wireless.

His research interests include wireless network security and privacy preservation. He is an Associate Editor for Peer-to-Peer Networking and Applications.



Yong Yu (Senior Member, IEEE) received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2008. He is currently a Professor with Shaanxi Normal University, Xi'an. He holds the Prestigious One Hundred Talent Professorship of Shaanxi Province. He has authored over 100 referred journal articles and conference papers. His research interests are blockchain and cloud security.